Task 1: Install Terraform on Windows PC

1. Objective

To install Terraform on a Windows PC, configure the system environment PATH, and verify the installation using Git Bash.

2. Purpose

Terraform is an Infrastructure as Code (IaC) tool developed by HashiCorp. It enables DevOps engineers to define, provision, and manage infrastructure using configuration files.

Installing Terraform is the first step toward automating infrastructure deployment and integrating Terraform with Jenkins CI/CD pipelines.

3. Steps Used

Step 1: Download Terraform

1. Opened the official HashiCorp installation page: https://developer.hashicorp.com/terraform/install
2. Selected the Windows AMD64 package for the 64-bit Windows operating system.
3. Downloaded the Terraform ZIP archive.

Step 2: Extract Terraform

1. Created the folder `C:\Terraform`.
2. Extracted the downloaded ZIP archive into the folder.
3. Verified that the executable file `terraform.exe` was available at `C:\Terraform\terraform.exe`.

Step 3: Configure the PATH environment variable

1. Opened Windows Search and searched for Edit the system environment variables.
2. Opened the Environment Variables window.
3. Selected the `Path` variable under User variables.
4. Added the following entry:

```
C:\Terraform
```

5. Saved the changes and reopened Git Bash to load the updated environment.

Step 4: Verify the Terraform installation

Executed the following command in Git Bash:

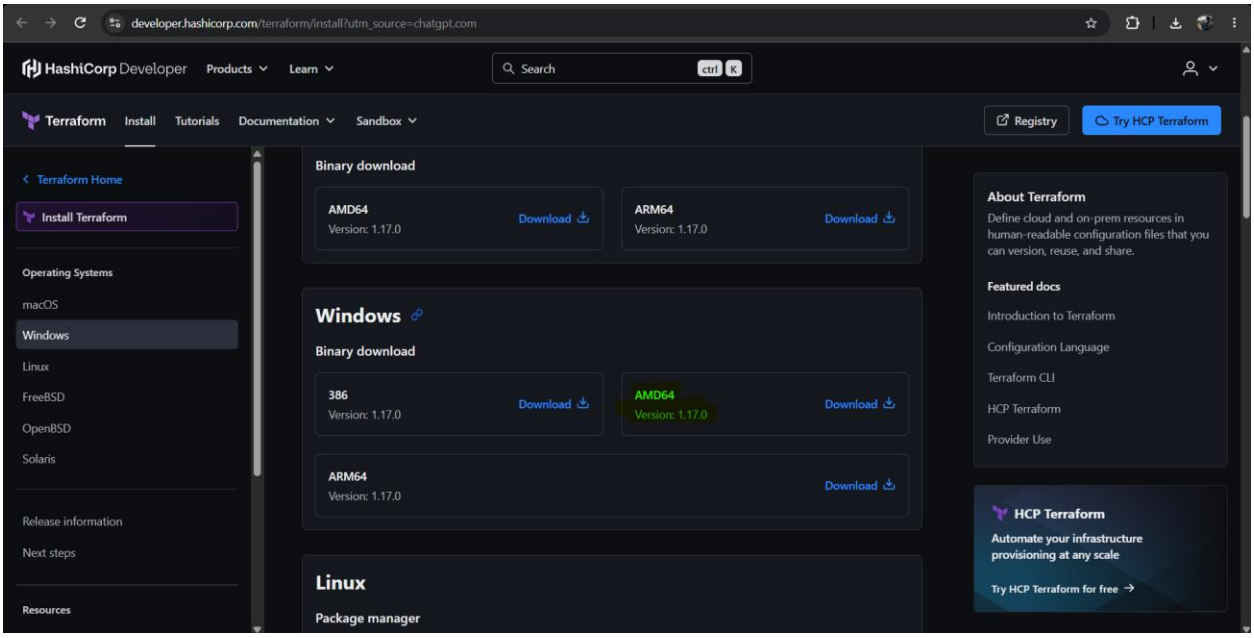
```
terraform -version
```

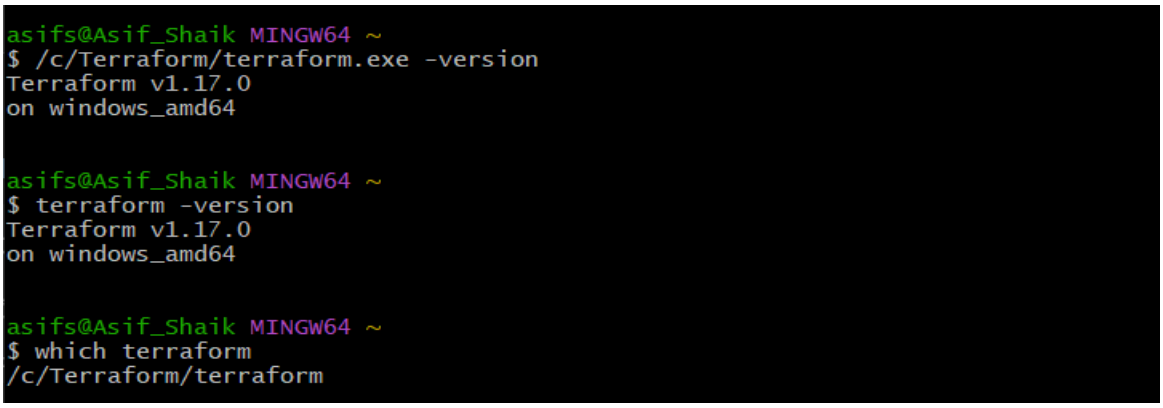
Step 5: Verify the executable location

Executed the command:

```
which terraform
```

This command was used to verify that Git Bash could locate the Terraform executable through PATH.





4. Observation

- Terraform was successfully extracted into `C:\Terraform`.
- Initially, Git Bash returned `command not found` because the executable was not accessible through PATH.
- The executable was verified directly using:

```
/c/Terraform/terraform.exe -version
```

- Terraform was identified as version `1.17.0` for Windows AMD64.
- After configuring PATH, the following command executed successfully:

```
terraform -version
```

Observed output:

```
Terraform v1.17.0
on windows_amd64
```

This confirmed that Terraform was installed correctly and accessible directly from Git Bash.

5. Conclusion

Terraform v1.17.0 was successfully installed on the Windows PC and verified using Git Bash. The Terraform executable was extracted to `C:\Terraform`, and the PATH configuration was corrected to make the executable accessible through the `terraform` command.

Task 2: Create and Execute Terraform Templates

1. Objective

To create and execute Terraform configuration templates in Visual Studio Code using the Local and Random providers.

2. Purpose

Terraform is an Infrastructure as Code (IaC) tool used to define and manage resources through configuration files. Terraform providers enable interaction with different platforms and services.

This practical demonstrates how to create a local text file and generate a random pet name using Terraform. It also covers the Terraform workflow, including initialization, formatting, validation, planning, applying, and state verification.

3. Steps Used

Step 1: Create the Terraform project

1. Opened Visual Studio Code.
2. Created and opened the project folder `Terraform-01`.
3. Created a configuration file named `main.tf`.
4. Added the Terraform configuration using the Local and Random providers.
5. Saved the file.

Step 2: Configure Terraform resources

The configuration contained two resources:

- `local_file.asif_pet` — creates `asif_pets.txt` with the content `My pet name is Brownie - Asif Shaik`.
- `random_pet.brownie` — generates a random pet-style name using the prefix `brownie`, a dot separator, and a length of three.

Step 3: Initialize Terraform

Executed:

```
terraform init
```

This initialized the working directory and downloaded the required provider plugins.

Step 4: Format and validate the configuration

Executed:

```
terraform fmt
terraform validate
```

Terraform formatted the configuration and validated its syntax and configuration structure.

Step 5: Review the Terraform plan

Executed:

```
terraform plan
```

The plan showed:

```
Plan: 2 to add, 0 to change, 0 to destroy.
```

Step 6: Apply the configuration

Executed:

```
terraform apply
```

Confirmed the operation by typing `yes`.

Terraform created both resources successfully.

Step 7: Verify the generated pet name

Executed:

```
terraform state show random_pet.brownie
```

The generated name was:

```
brownie.really.engaging.haddock
```

Step 8: Verify the Terraform state

Executed:

```
terraform state list
```

The output confirmed the following resources:

```
local_file.asif_pet
random_pet.brownie
```

Step 9: Verify the local file

The expected file was `asif_pets.txt`, containing:

```
My pet name is Brownie - Asif Shaik
```

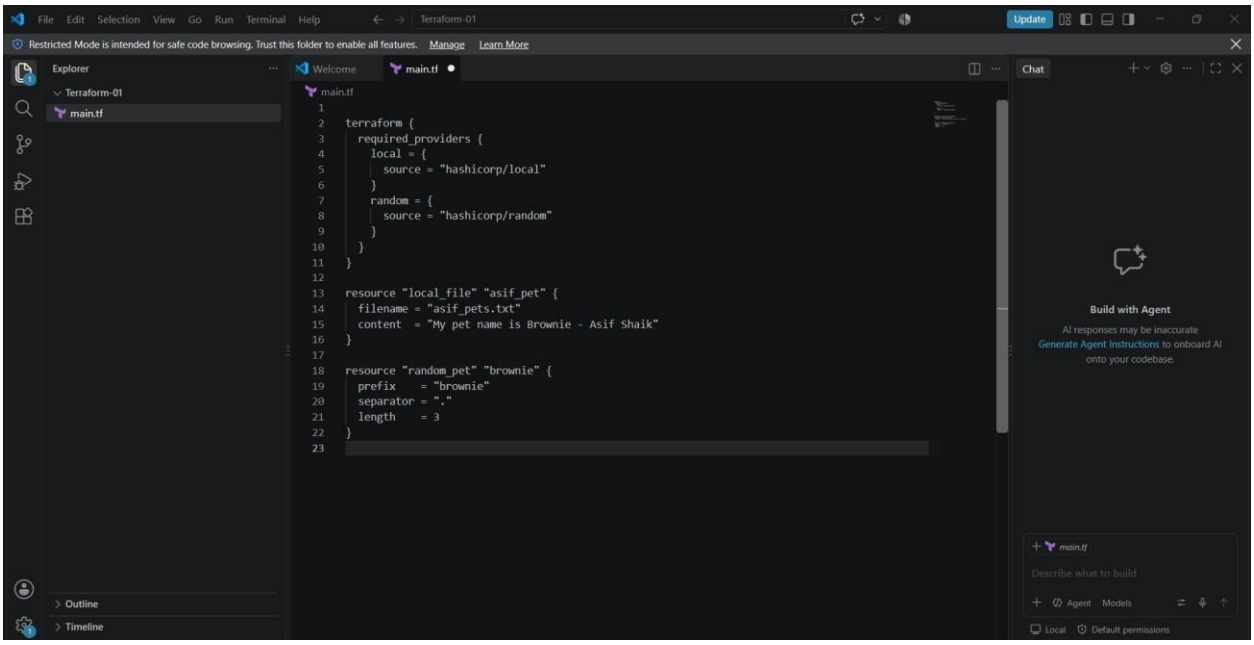
The file can be verified using:

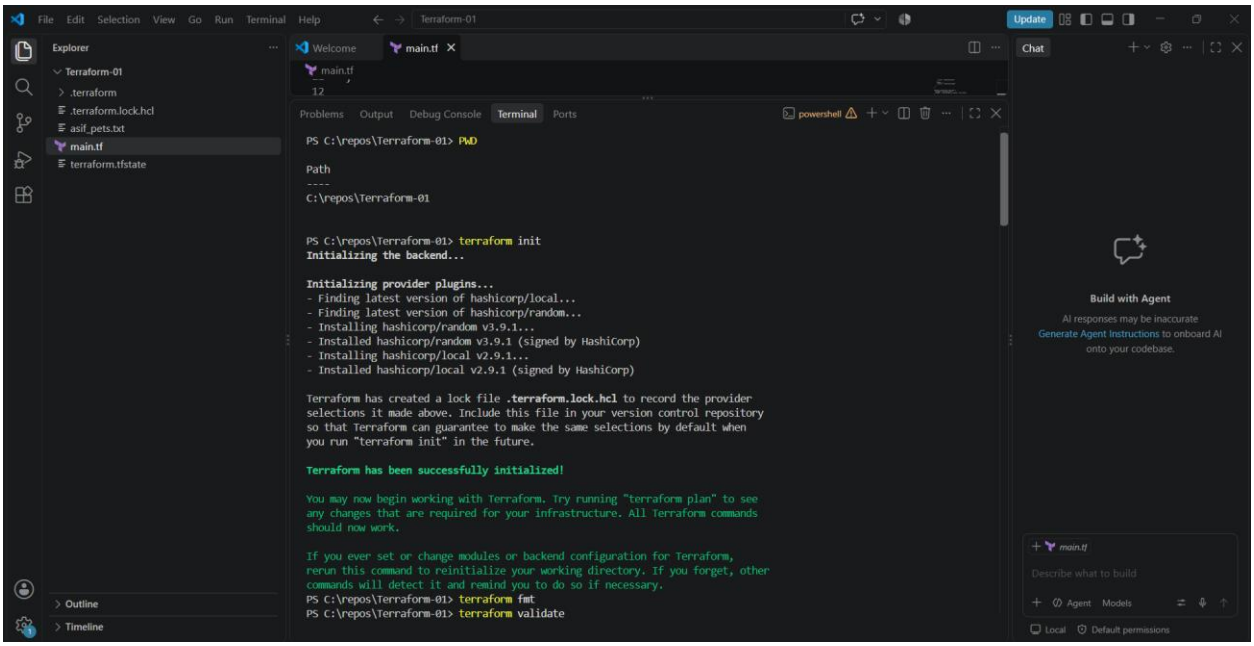
```
Get-Content asif_pets.txt
```

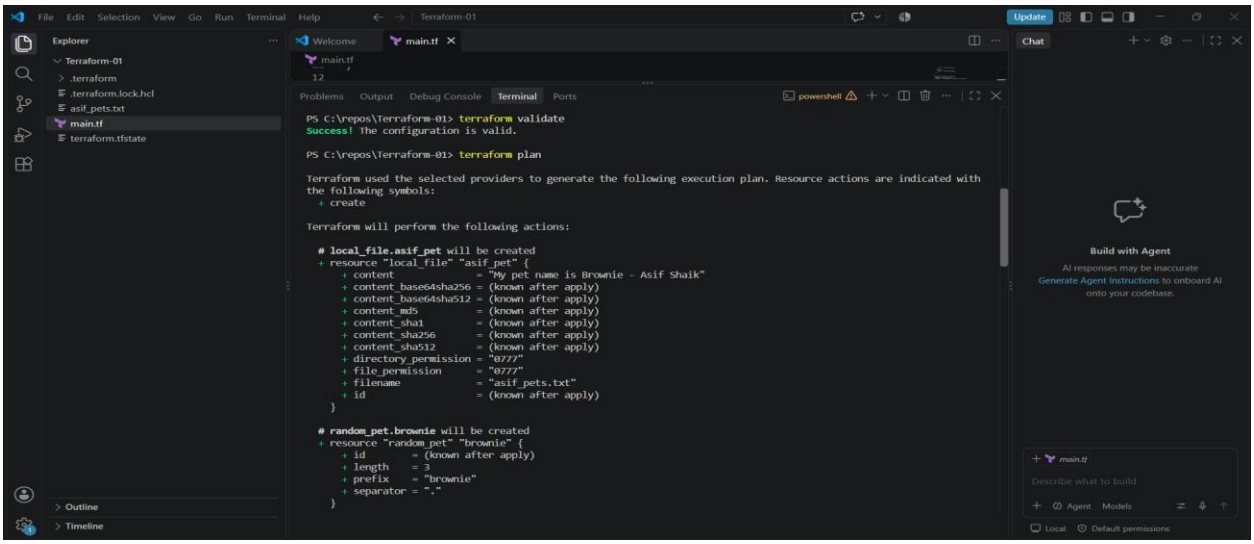
```
terraform {
  required_providers {
    local = {
      source = "hashicorp/local"
    }
    random = {
      source = "hashicorp/random"
    }
  }
}

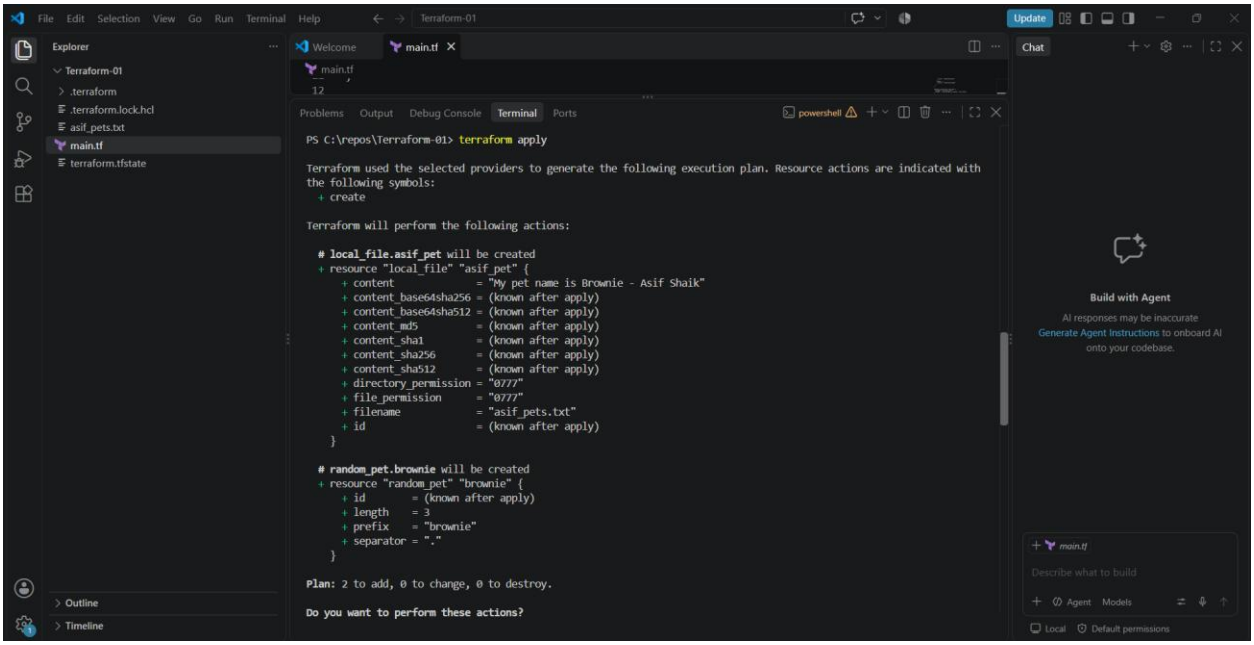
resource "local_file" "asif_pet" {
  filename = "asif_pets.txt"
  content  = "My pet name is Brownie - Asif Shaik"
}
```

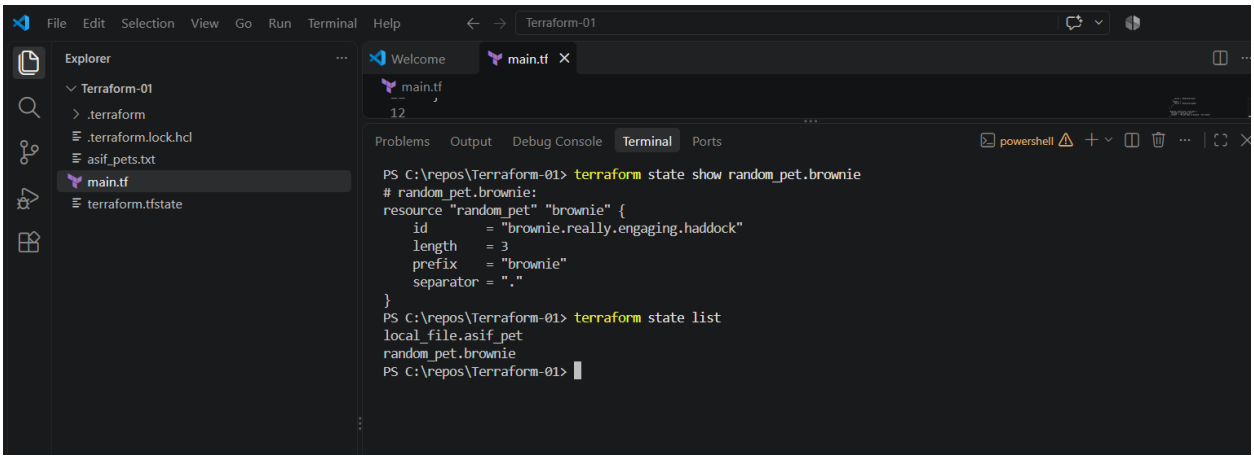
```
resource "random_pet" "brownie" {
  prefix    = "brownie"
  separator = "."
  length    = 3
}
```






File Edit Selection View Go Run Terminal Help
Terraform-01
Update
Explorer
Terraform-01
.terraform
.terraform.lock.hcl
asif_pets.txt
main.tf
terraform.tfstate
Welcome
main.tf
Problems Output Debug Console Terminal Ports
powershell
PS C:\repos\Terraform-01> terraform validate
Success! The configuration is valid.
PS C:\repos\Terraform-01> terraform plan
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
Terraform will perform the following actions:
local_file.asif_pet will be created
+ resource "local_file" "asif_pet" {
+ content = "My pet name is Brownie - Asif Shaik"
+ content_base64sha256 = (known after apply)
+ content_base64sha512 = (known after apply)
+ content_md5 = (known after apply)
+ content_sha1 = (known after apply)
+ content_sha256 = (known after apply)
+ content_sha512 = (known after apply)
+ directory_permission = "0777"
+ file_permission = "0777"
+ filename = "asif_pets.txt"
+ id = (known after apply)
}
random_pet.brownie will be created
+ resource "random_pet" "brownie" {
+ id = (known after apply)
+ length = 3
+ prefix = "brownie"
+ separator = "."
}
Outline
Timeline
Chat
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
main.tf
Describe what to build
Agent Models
Local Default permissions


File Edit Selection View Go Run Terminal Help
Terraform-01
Update
Explorer
Terraform-01
.terraform
.terraform.lock.hcl
asif_pets.txt
main.tf
terraform.tfstate
Welcome
main.tf
Problems Output Debug Console Terminal Ports
powershell
PS C:\repos\Terraform-01> terraform apply
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
Terraform will perform the following actions:
local_file.asif_pet will be created
+ resource "local_file" "asif_pet" {
+ content = "My pet name is Brownie - Asif Shaik"
+ content_base64sha256 = (known after apply)
+ content_base64sha512 = (known after apply)
+ content_md5 = (known after apply)
+ content_sha1 = (known after apply)
+ content_sha256 = (known after apply)
+ content_sha512 = (known after apply)
+ directory_permission = "0777"
+ file_permission = "0777"
+ filename = "asif_pets.txt"
+ id = (known after apply)
}
random_pet.brownie will be created
+ resource "random_pet" "brownie" {
+ id = (known after apply)
+ length = 3
+ prefix = "brownie"
+ separator = "."
}
Plan: 2 to add, 0 to change, 0 to destroy.
Do you want to perform these actions?
Outline
Timeline
Chat
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
main.tf
Describe what to build
Agent Models
Local Default permissions



4. Observation

- Terraform initialized successfully.
- The required Local and Random providers were downloaded.
- The configuration was formatted and validated.
- The Terraform plan identified two resources to create.
- Both resources were created successfully using `terraform apply`.
- The Random provider generated the name `brownie.really.engaging.haddock`.
- Terraform state listed both resources, confirming that they are managed by Terraform.
- The Local provider was configured to create the file `asif_pets.txt` with the specified content.

5. Conclusion

The Terraform configuration was successfully created and executed in Visual Studio Code. The Local provider was used to create a text file, while the Random provider generated a pet-style name for Brownie.

This practical demonstrated the basic Terraform workflow: initialization, formatting, validation, planning, applying, and state verification. It established a foundation for understanding Terraform providers and infrastructure automation.

Task 3: Terraform Fundamentals — Init, Plan, Apply and Provider

1. Terraform Init

Command: `terraform init`

Purpose: Initializes the Terraform working directory and prepares it for Terraform operations. It downloads the required provider plugins, installs modules, and initializes the configured backend.

Example:

```
cd C:\repos\Terraform-01
terraform init
```

Expected result:

```
Initializing the backend...
Initializing provider plugins...
Terraform has been successfully initialized!
```

Explanation: When starting a new Terraform project, Terraform needs the required providers and dependencies before it can process the configuration. The `terraform init` command prepares the working directory for further operations.

Key point: This command prepares the environment but does not create infrastructure resources.

2. Terraform Plan

Command: `terraform plan`

Purpose: Reviews the Terraform configuration and determines what changes are required to reach the desired infrastructure state. It generates a plan without applying the changes.

Example:

```
resource "local_file" "example" {
  filename = "example.txt"
  content  = "Hello from Terraform"
}
```

Run the following command:

```
terraform plan
```

Expected result: Terraform displays a plan showing that it intends to create one local file.

```
Plan: 1 to add, 0 to change, 0 to destroy.
```

Explanation: Terraform checks the configuration against its current state and available resource information to determine the proposed changes. The plan helps identify potential changes before execution.

Key point: `terraform plan` previews changes; it does not apply them.

3. Terraform Apply

Command: `terraform apply`

Purpose: Executes the infrastructure changes defined in the Terraform configuration.

Example: Using the same `local_file` resource from the previous section, run:

```
terraform apply
```

Terraform displays the proposed changes and generally requests confirmation:

```
Do you want to perform these actions?
  Enter a value: yes
```

After you enter `yes`, Terraform creates the file.

Expected result:

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

The file `example.txt` will contain:

```
Hello from Terraform
```

Explanation: Terraform carries out the planned operation and updates its state to record the managed resource.

Key point: `terraform apply` makes the actual changes. Review the plan before approving it, especially when working with cloud infrastructure that may incur charges.

4. Terraform Provider

Definition: A Terraform provider is a plugin that allows Terraform to communicate with and manage resources on a specific platform or service.

Examples include:

- AWS provider — manages AWS resources such as EC2 instances and S3 buckets.
- Azure provider — manages Azure resources.
- Docker provider — manages Docker containers and images.
- Local provider — manages local files and directories.

Example: AWS provider configuration

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}
```

Explanation:

- `required_providers` identifies the provider Terraform must install.
- `source` specifies where the provider comes from.
- `version` defines the acceptable provider version range.
- `provider "aws"` configures the AWS provider.
- `region` specifies the AWS Region in which regional resources are managed.

After configuring the provider, run:

```
terraform init
```

Terraform downloads the AWS provider plugin that satisfies the version constraint.

Key point: Providers enable Terraform to interact with external platforms and services. The provider itself does not automatically create resources; resource blocks define what Terraform should manage.

5. Complete Terraform Workflow

The following commands represent the typical workflow:

```
terraform init
terraform plan
terraform apply
```

Command	Main purpose	Creates infrastructure?
`terraform init`	Initializes the working directory and dependencies	No
`terraform plan`	Previews proposed changes	No
`terraform apply`	Executes infrastructure changes	Yes, when the configuration requires it
Provider configuration	Specifies how Terraform connects to a platform	Not by itself

Overall Conclusion

Terraform is an Infrastructure as Code (IaC) tool that enables infrastructure to be defined, reviewed, and managed through configuration files. The `terraform init` command initializes the working directory and downloads required dependencies. The `terraform plan` command previews the proposed changes so they can be reviewed before execution. The `terraform apply` command executes the required changes and updates Terraform state. Providers connect Terraform to platforms such as AWS, Azure, Docker, and local systems, allowing it to manage their resources.

Task 4: Integrate a Sample Terraform Template in Jenkins

1. Objective

To integrate Terraform with Jenkins and automate infrastructure-as-code operations using a Jenkins Declarative Pipeline. The pipeline performs Terraform version verification, initialization, configuration validation, planning, application, and output verification.

The task uses Terraform's local provider to create a sample text file without provisioning AWS infrastructure resources.

2. Steps Performed

Step 1: Accessed the Existing Jenkins Server

- Reused the existing AWS EC2 instance named `Jenkins-New-Server`.
- Connected to the Jenkins dashboard using the public IP address `3.80.35.212`.
- Confirmed that Jenkins was accessible.

Step 2: Verified the Environment

- Operating system: Amazon Linux 2023.
- Verified Terraform was initially not installed on the server.
- Installed Terraform using the official HashiCorp package repository.
- Verified the Terraform version as `1.17.0`.
- Verified that Git was installed, version `2.50.1`.

Step 3: Verified Terraform Access for Jenkins

Executed the following command:

```
sudo -u jenkins terraform --version
```

Result: Confirmed that the Jenkins service account could execute Terraform successfully.

Step 4: Created the Jenkins Pipeline Job

- Opened the Jenkins dashboard.
- Selected **New Item**.

- Created a Pipeline job named `Terraform-Demo-Pipeline`.
- Selected the Pipeline project type.

Step 5: Created the Terraform Configuration

Created a sample Terraform configuration named `main.tf` within the Jenkins workspace.

The configuration uses the `hashicorp/local` provider to create a text file named `jenkins-terraform-demo.txt`.

Step 6: Configured the Jenkins Pipeline

Added the Terraform stages to the Declarative Pipeline:

1. Terraform Version
2. Create Terraform Template
3. Terraform Init
4. Terraform Validate
5. Terraform Plan
6. Terraform Apply
7. Verify Output File

Step 7: Executed the Pipeline

- Saved the pipeline configuration.
- Selected **Build Now**.
- Opened **Console Output** to monitor the execution.
- Resolved the initial workspace permission error by generating the Terraform configuration inside the Jenkins workspace rather than accessing the `ec2-user` home directory.

Step 8: Verified the Result

The pipeline completed successfully and created the sample text file in the Jenkins workspace.

Expected file content:

```
Terraform executed through Jenkins successfully!
```

3. Complete Jenkins Pipeline

```
pipeline {
    agent any

    stages {
        stage('Terraform Version') {
           steps {
                sh 'terraform --version'
            }
        }

        stage('Create Terraform Template') {
            steps {
                sh '''
                    cat > main.tf <<'EOF'
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.0"
    }
  }
}

resource "local_file" "jenkins_demo" {
  filename = "${path.module}/jenkins-terraform-demo.txt"
  content  = "Terraform executed through Jenkins successfully!"
}
EOF
                '''
            }
        }

        stage('Terraform Init') {
            steps {
                sh 'terraform init'
            }
        }
```

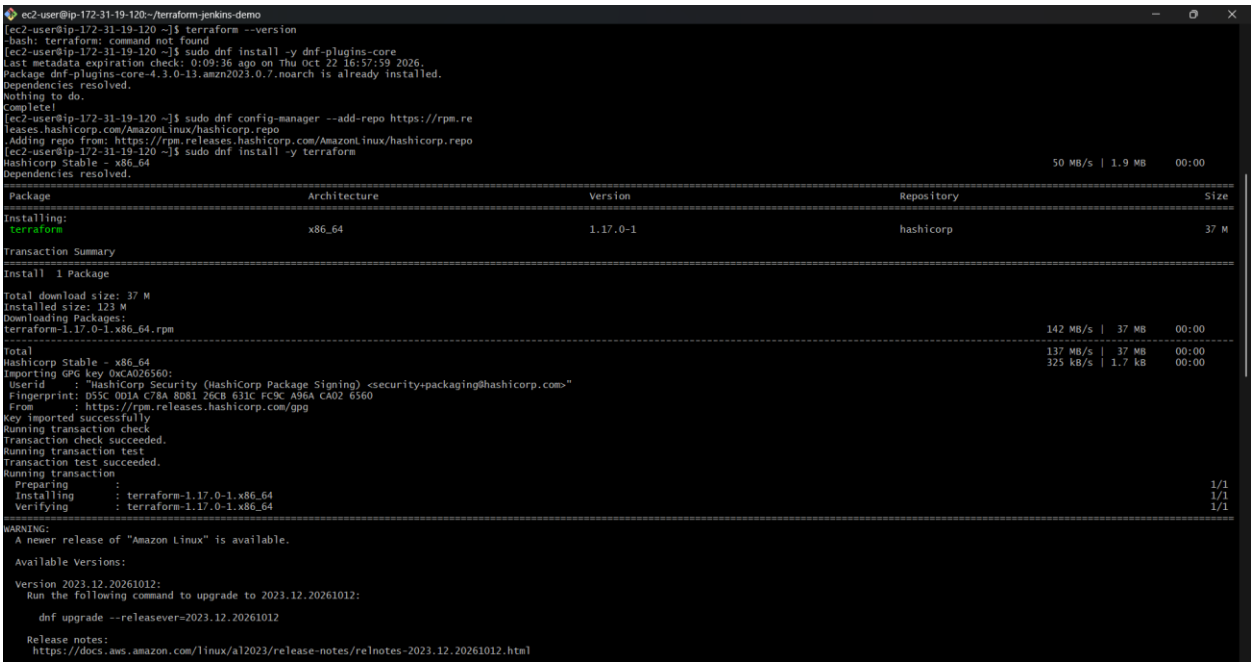
```
        stage('Terraform Validate') {
            steps {
                sh 'terraform validate'
            }
        }

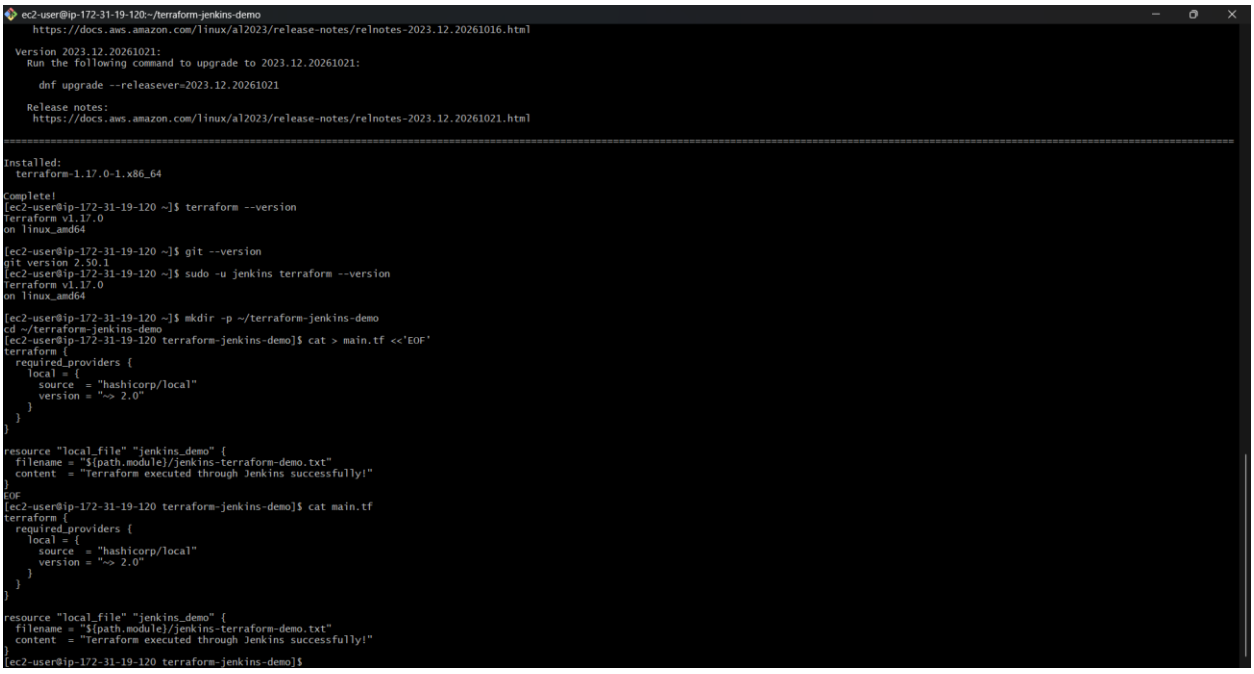
        stage('Terraform Plan') {
            steps {
                sh 'terraform plan'
            }
        }

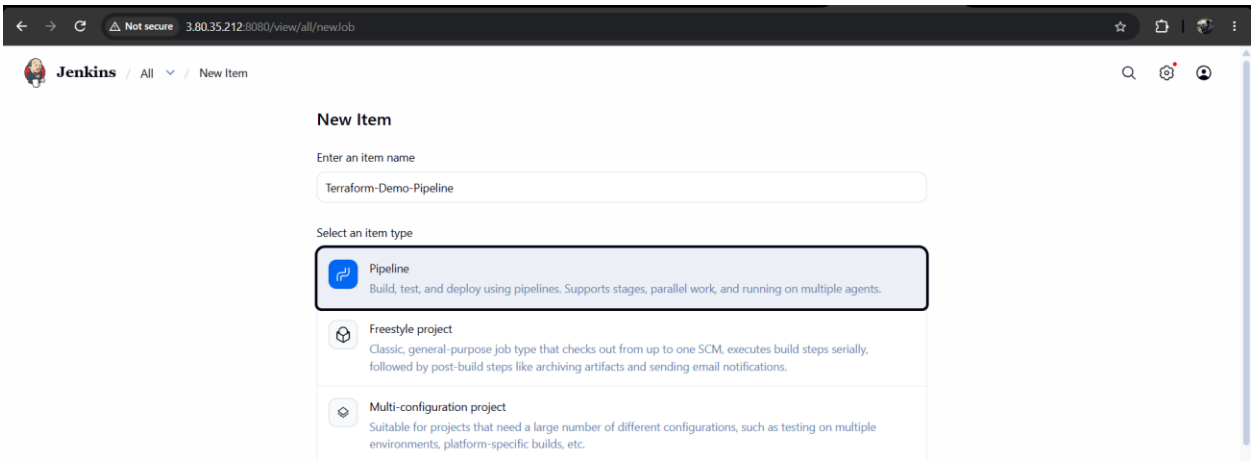
        stage('Terraform Apply') {
            steps {
                sh 'terraform apply -auto-approve'
            }
        }

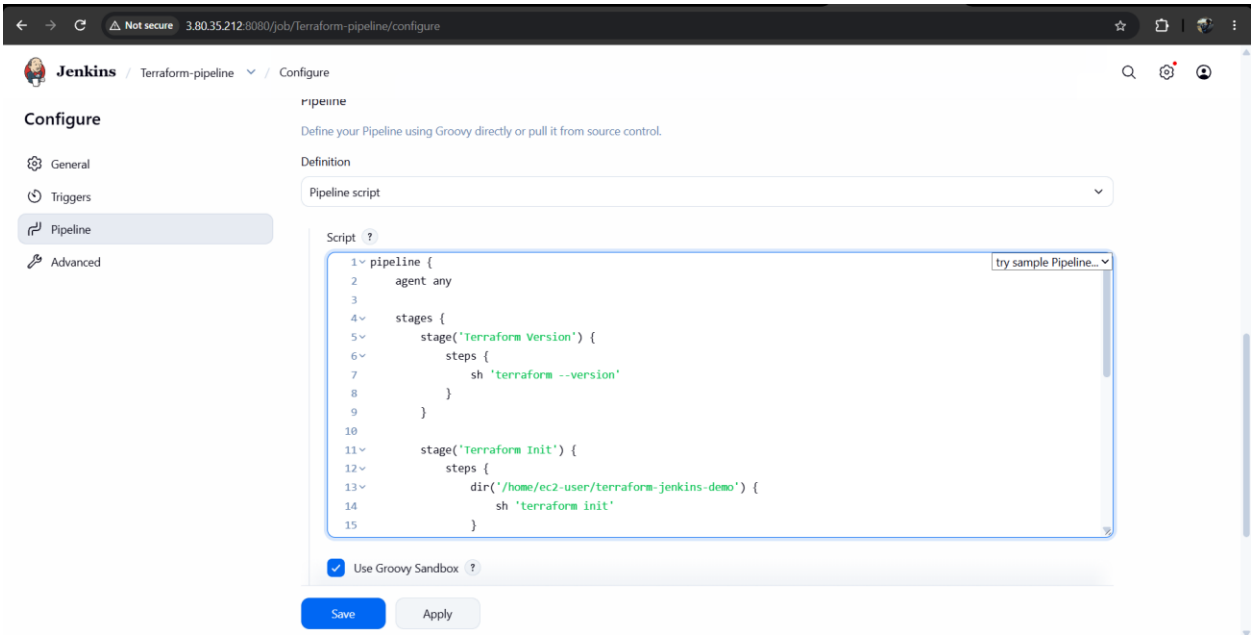
        stage('Verify Output File') {
            steps {
                sh '''
                    echo "Checking Terraform-created file..."
                    cat jenkins-terraform-demo.txt
                '''
            }
        }
    }

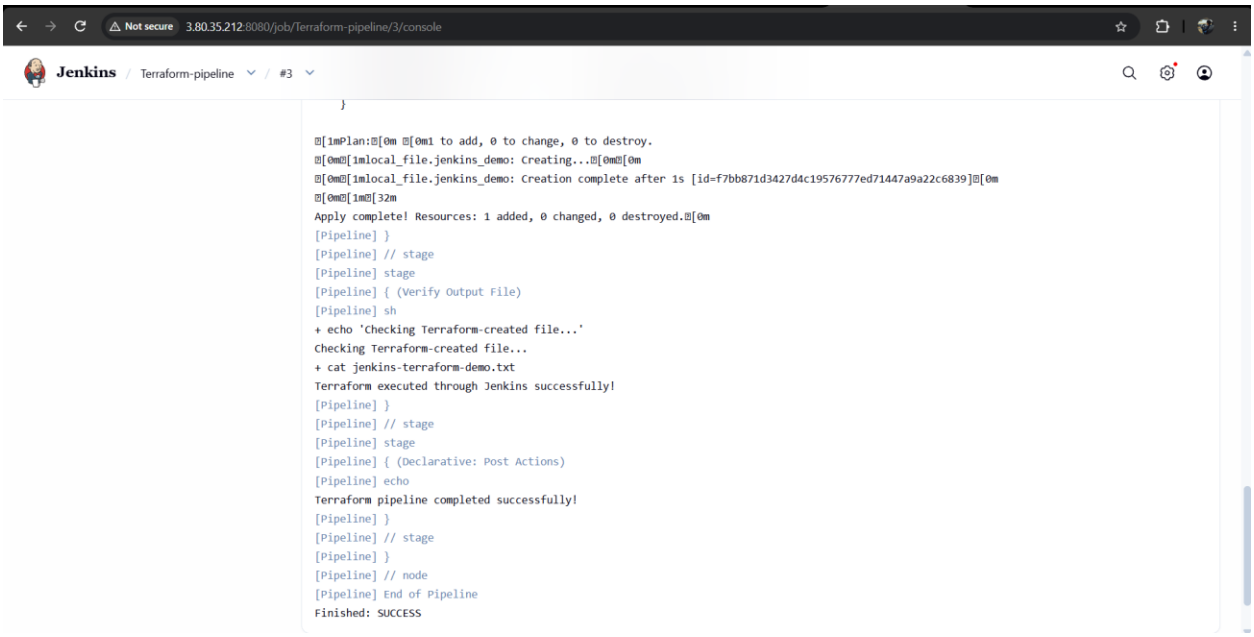
    post {
        success {
            echo 'Terraform pipeline completed successfully!'
        }
        failure {
            echo 'Terraform pipeline failed. Check the console
output.'
        }
    }
}
```

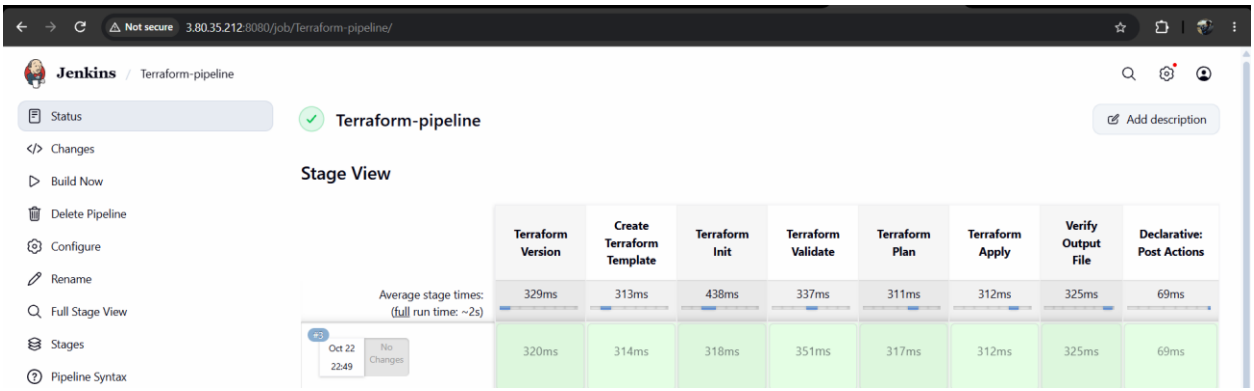

ec2-user@ip-172-31-19-120:~/terraform-jenkins-demo
[ec2-user@ip-172-31-19-120 ~]$ terraform --version
-bash: terraform: command not found
[ec2-user@ip-172-31-19-120 ~]$ sudo dnf install -y dnf-plugins-core
Last metadata expiration check: 0:09:36 ago on Thu Oct 22 16:57:59 2026.
Package dnf-plugins-core-4.3.0-13.amzn2023.0.7.noarch is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-19-120 ~]$ sudo dnf config-manager --add-repo https://rpm.re
leases.hashicorp.com/AmazonLinux/hashicorp.repo
Adding repo from: https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
[ec2-user@ip-172-31-19-120 ~]$ sudo dnf install -y terraform
Hashicorp Stable - x86_64
50 MB/s | 1.9 MB 00:00
Dependencies resolved.
Package Architecture Version Repository Size
Installing:
terraform x86_64 1.17.0-1 hashicorp 37 M
Transaction Summary
Install 1 Package
Total download size: 37 M
Installed size: 123 M
Downloading Packages:
terraform-1.17.0-1.x86_64.rpm
142 MB/s | 37 MB 00:00
Total
137 MB/s | 37 MB 00:00
Hashicorp Stable - x86_64
325 kB/s | 1.7 kB 00:00
Importing GPG key 0xCA026560:
Userid : "HashiCorp Security (HashiCorp Package Signing) <security+packaging@hashicorp.com>"
Fingerprint: D55C 0D1A C78A 8D81 26CB 631C FC9C A96A CA02 6560
From : https://rpm.releases.hashicorp.com/gpg
Key imported successfully
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
Preparing :
Installing : terraform-1.17.0-1.x86_64
Verifying : terraform-1.17.0-1.x86_64
1/1
1/1
1/1
WARNING:
A newer release of "Amazon Linux" is available.
Available Versions:
Version 2023.12.20261012:
Run the following command to upgrade to 2023.12.20261012:
dnf upgrade --releasever=2023.12.20261012
Release notes:
https://docs.aws.amazon.com/linux/al2023/release-notes/relnotes-2023.12.20261012.html


ec2-user@ip-172-31-19-120:~/terraform-jenkins-demo
https://docs.aws.amazon.com/linux/al2023/release-notes/relnotes-2023.12.20261016.html
Version 2023.12.20261021:
Run the following command to upgrade to 2023.12.20261021:
dnf upgrade --releasever=2023.12.20261021
Release notes:
https://docs.aws.amazon.com/linux/al2023/release-notes/relnotes-2023.12.20261021.html
Installed:
terraform-1.17.0-1.x86_64
Complete!
[ec2-user@ip-172-31-19-120 ~]$ terraform --version
Terraform v1.17.0
on linux_amd64
[ec2-user@ip-172-31-19-120 ~]$ git --version
git version 2.50.1
[ec2-user@ip-172-31-19-120 ~]$ sudo -u jenkins terraform --version
Terraform v1.17.0
on linux_amd64
[ec2-user@ip-172-31-19-120 ~]$ mkdir -p ~/terraform-jenkins-demo
cd ~/terraform-jenkins-demo
[ec2-user@ip-172-31-19-120 terraform-jenkins-demo]$ cat > main.tf <<'EOF'
terraform {
required_providers {
local = {
source = "hashicorp/local"
version = "~> 2.0"
}
}
}
resource "local_file" "jenkins_demo" {
filename = "${path.module}/jenkins-terraform-demo.txt"
content = "Terraform executed through Jenkins successfully!"
}
EOF
[ec2-user@ip-172-31-19-120 terraform-jenkins-demo]$ cat main.tf
terraform {
required_providers {
local = {
source = "hashicorp/local"
version = "~> 2.0"
}
}
}
resource "local_file" "jenkins_demo" {
filename = "${path.module}/jenkins-terraform-demo.txt"
content = "Terraform executed through Jenkins successfully!"
}
[ec2-user@ip-172-31-19-120 terraform-jenkins-demo]$


Not secure 3.80.35.212:8080/view/all/newJob
Jenkins / All / New Item
New Item
Enter an item name
Terraform-Demo-Pipeline
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.


Not secure 3.80.35.212:8080/job/Terraform-pipeline/configure
Jenkins / Terraform-pipeline / Configure
Configure
General
Triggers
Pipeline
Advanced
Pipeline
Define your Pipeline using Groovy directly or pull it from source control.
Definition
Pipeline script
Script
try sample Pipeline...
pipeline {
agent any
stages {
stage('Terraform Version') {
steps {
sh 'terraform --version'
}
}
stage('Terraform Init') {
steps {
dir('/home/ec2-user/terraform-jenkins-demo') {
sh 'terraform init'
}
Use Groovy Sandbox
Save
Apply





4. Observation

- Terraform version `1.17.0` was detected successfully by Jenkins.
- Terraform initialized the working directory and installed the `hashicorp/local` provider, version `2.9.1`.
- `terraform validate` confirmed that the configuration was valid.
- `terraform plan` displayed the expected result: `1 to add, 0 to change, 0 to destroy`.
- `terraform apply -auto-approve` created the local text file.
- The verification stage displayed the expected file content.
- The final Jenkins build status was **SUCCESS**.
- An initial `AccessDeniedException` occurred because Jenkins could not access the project directory under `/home/ec2-user`. This was resolved by creating the Terraform configuration in the Jenkins workspace.

5. Troubleshooting Performed

Issue	Cause	Resolution
`terraform: command not found`	Terraform was not installed on the EC2 server	Added the HashiCorp repository and installed Terraform
Jenkins could not access the project directory	The Jenkins service account lacked permission to access `/home/ec2-user/terraform-jenkins-demo`	Moved the configuration-generation step into the Jenkins workspace
Validation and plan stages were skipped in the initial build	The pipeline failed during the directory-access step	Updated the pipeline and reran the build successfully

6. Conclusion

The integration of Terraform with Jenkins was completed successfully using a Declarative Pipeline on an existing Amazon Linux 2023 EC2 server. Terraform was installed and made accessible to the Jenkins service account. A sample Terraform template was created using the local provider, and Jenkins automated the initialization, validation, planning, and application stages.

The pipeline successfully created and verified the expected text file in the Jenkins workspace. An initial workspace permission issue was identified and resolved by generating the configuration within the Jenkins workspace. This task demonstrated how Jenkins can automate Terraform operations through a repeatable CI/CD workflow without provisioning additional AWS infrastructure resources.